

Approximate Training

:label: sec_approx_train

Recall our discussions in :numref: sec_word2vec. The main idea of the skip-gram model is using softmax operations to calculate the conditional probability of generating a context word w_o based on the given center word w_c in :eqref: eq_skip-gram-softmax, whose corresponding logarithmic loss is given by the opposite of :eqref: eq_skip-gram-log.

Due to the nature of the softmax operation, since a context word may be anyone in the dictionary $\mathcal{V}$, the opposite of :eqref: eq_skip-gram-log contains the summation of items as many as the entire size of the vocabulary. Consequently, the gradient calculation for the skip-gram model in :eqref: eq_skip-gram-grad and that for the continuous bag-of-words model in :eqref: eq_cbow-gradient both contain the summation. Unfortunately, the computational cost for such gradients that sum over a large dictionary (often with hundreds of thousands or millions of words) is huge!

In order to reduce the aforementioned computational complexity, this section will introduce two approximate training methods: *negative sampling* and *hierarchical softmax*. Due to the similarity between the skip-gram model and the continuous bag of words model, we will just take the skip-gram model as an example to describe these two approximate training methods.

Negative Sampling

:label: subsec_negative-sampling

Negative sampling modifies the original objective function. Given the context window of a center word w_c , the fact that any (context) word w_o comes from this context window is considered as an event with the probability modeled by

$$P(D=1 \mid w_c, w_o) = \sigma(\mathbf{u}_o^\top \mathbf{v}_c),$$

where σ uses the definition of the sigmoid activation function:

$$\sigma(x) = \frac{1}{1 + \exp(-x)}.$$
 :eqlabel: eq_sigma-f

Let's begin by maximizing the joint probability of all such events in text sequences to train word embeddings. Specifically, given a text sequence of length T , denote by $w^{(t)}$ the word at time step t and let the context window size be m , consider maximizing the joint probability

$$\prod_{t=1}^T \prod_{-m \leq j \leq m, j \neq 0} P(D=1 \mid w^{(t)}, w^{(t+j)})$$

:eqlabel: eq-negative-sample-pos

However, :eqref: eq-negative-sample-pos only considers those events that involve positive examples. As a result, the joint probability in :eqref: eq-negative-sample-pos is maximized to 1 only if all the word vectors are equal to infinity. Of course, such results are meaningless. To make the objective function more meaningful, *negative sampling* adds negative examples sampled from a predefined distribution.

Denote by S the event that a context word w_o comes from the context window of a center word w_c . For this event involving w_o , from a predefined distribution $P(w)$ sample K *noise words* that are not from this context window. Denote by N_k the event that a noise word w_k ($k=1, \dots, K$) does not come from the context window of w_c . Assume that these events involving both the positive example and negative examples $S, N_1, \dots, N_K$ are mutually independent. Negative sampling rewrites the joint probability (involving only positive examples) in :eqref: eq-negative-sample-pos as

$$\prod_{t=1}^T \prod_{-m \leq j \leq m, j \neq 0} P(w^{(t+j)} \mid w^{(t)}),$$

where the conditional probability is approximated through events $S, N_1, \dots, N_K$:

$$P(w^{(t+j)} \mid w^{(t)}) = P(D=1 \mid w^{(t)}, w^{(t+j)}) \prod_{k=1}^K P(w_k \sim P(w))^{N_k} P(D=0 \mid w^{(t)}, w_k)$$

:eqlabel: eq-negative-sample-conditional-prob

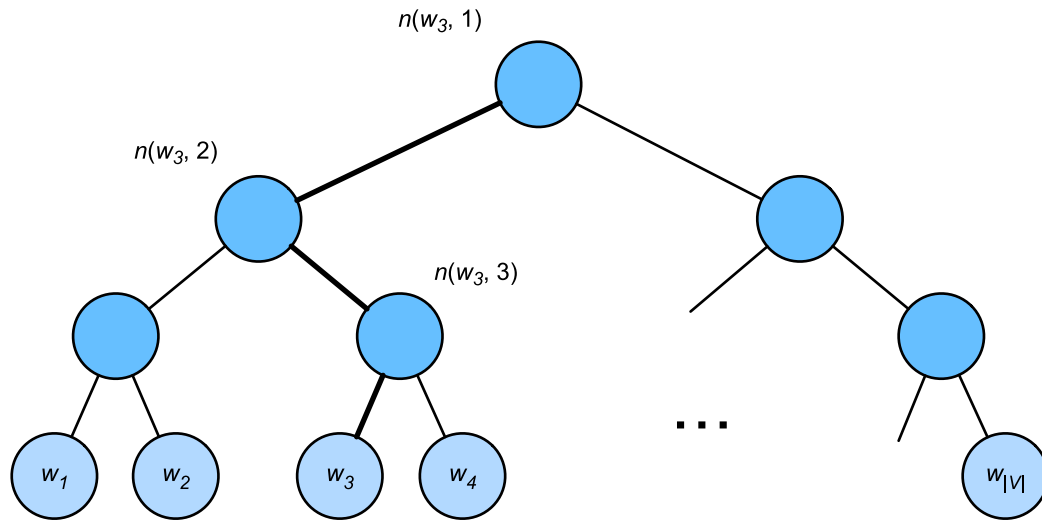
Denote by i_t and h_k the indices of a word $w^{(t)}$ at time step t of a text sequence and a noise word w_k , respectively. The logarithmic loss with respect to the conditional probabilities in :eqref: eq-negative-sample-conditional-prob is

$$\begin{aligned} -\log P(w^{(t+j)} \mid w^{(t)}) &= -\log P(D=1 \mid w^{(t)}, w^{(t+j)}) \\ &- \sum_{k=1}^K \log P(D=0 \mid w^{(t)}, w_k) \\ &= -\log \left(\frac{P(w^{(t+j)})}{\sum_{k=1}^K P(w_k) + P(w^{(t+j)})} \right) \\ &= -\log \left(\frac{P(w^{(t+j)})}{\sum_{k=1}^K P(w_k) + P(w^{(t+j)})} \right) \\ &= -\log \left(\frac{P(w^{(t+j)})}{\sum_{k=1}^K P(w_k) + P(w^{(t+j)})} \right) \end{aligned}$$

We can see that now the computational cost for gradients at each training step has nothing to do with the dictionary size, but linearly depends on K . When setting the hyperparameter K to a smaller value, the computational cost for gradients at each training step with negative sampling is smaller.

Hierarchical Softmax

As an alternative approximate training method, *hierarchical softmax* uses the binary tree, a data structure illustrated in :numref:fig_hi_softmax, where each leaf node of the tree represents a word in dictionary $\mathcal{V}$.



:label:fig_hi_softmax

Denote by $L(w)$ the number of nodes (including both ends) on the path from the root node to the leaf node representing word w in the binary tree. Let $n(w, j)$ be the j^{th} node on this path, with its context word vector being $\mathbf{u}_{n(w, j)}$. For example, $L(w_3) = 4$ in :numref:fig_hi_softmax.

Hierarchical softmax approximates the conditional probability in :eqref:eq_skip-gram-softmax as

$$P(w_o \mid w_c) = \prod_{j=1}^{L(w_o)-1} \sigma(\mathbf{u}_{n(w_o, j+1)} - \mathbf{v}_c) \cdot \prod_{j=1}^{L(w_o)-1} \sigma(\mathbf{u}_{n(w_o, j)} - \mathbf{v}_c)$$

where function σ is defined in :eqref:eq_sigma-f, and $\mathbf{u}_{n(w_o, j)}$ is the left child node of node n : if x is true, $\sigma(x) = 1$; otherwise $\sigma(x) = -1$.

To illustrate, let's calculate the conditional probability of generating word w_3 given word w_c in :numref:fig_hi_softmax. This requires dot products between the word vector $\mathbf{v}_c$ of w_c and non-leaf node vectors on the path (the path in bold in :numref:fig_hi_softmax) from the root to w_3 , which is traversed left, right, then left:

$$P(w_3 \mid w_c) = \sigma(\mathbf{u}_{n(w_3, 1)} - \mathbf{v}_c) \cdot \sigma(\mathbf{u}_{n(w_3, 2)} - \mathbf{v}_c) \cdot \sigma(\mathbf{u}_{n(w_3, 3)} - \mathbf{v}_c)$$

Since $\sigma(x) + \sigma(-x) = 1$, it holds that the conditional probabilities of generating all the words in dictionary $\mathcal{V}$ based on any word w_c sum up to one:

$$\sum_{w \in \mathcal{V}} P(w \mid w_c) = 1. \quad \text{:eqref: eq_hi-softmax-sum-one}$$

Fortunately, since $L(w_o) - 1$ is on the order of $\mathcal{O}(\log_2 |\mathcal{V}|)$ due to the binary tree structure, when the dictionary size $|\mathcal{V}|$ is huge, the computational cost for each training step using hierarchical softmax is significantly reduced compared with that without approximate training.

Summary

- Negative sampling constructs the loss function by considering mutually independent events that involve both positive and negative examples. The computational cost for training is linearly dependent on the number of noise words at each step.
- Hierarchical softmax constructs the loss function using the path from the root node to the leaf node in the binary tree. The computational cost for training is dependent on the logarithm of the dictionary size at each step.

Exercises

1. How can we sample noise words in negative sampling?
2. Verify that :eqref: eq_hi-softmax-sum-one holds.
3. How to train the continuous bag of words model using negative sampling and hierarchical softmax, respectively?

[Discussions](#)